

Ros Day03 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.11

목차

1. 과제 개요
2. 시스템 구성
3. Node 및 Topic 통신 구조
4. 신호등 상태 관리
5. 차량 이동 처리
6. YAML Parameter 구성
7. Launch 파일 구성
8. Qt 화면 구성
9. 결과 및 고찰

1. 과제 개요

이번 과제에서는 ROS2 의 Topic 통신과 Qt GUI 를 이용하여 신호등 및 차량이 동작하는 교차로 시뮬레이터를 제작하였다.

시뮬레이터는 신호등 상태를 관리하는 traffic_light_node, 차량의 이동을 처리하는 vehicle_node, 현재 신호와 차량 위치를 화면에 표시하는 Qt GUI 로 구성하였다.

신호등의 점등 시간과 차량의 이동 속도는 소스 코드에 직접 작성하지 않고 ROS2 Parameter 로 선언하였다. Parameter 의 실제 값은 YAML 파일에서 관리하고, Launch 파일을 이용하여 전체 Node 와 설정 파일을 한 번에 실행하도록 구성하였다.

2. 시스템 구성

전체 시스템은 기능에 따라 다음과 같이 분리하였다.

2.1 Traffic Light Node

traffic_light_node 는 신호등의 현재 상태를 관리한다.

신호는 다음 순서로 반복된다.

RED → GREEN → YELLOW → RED

각 신호의 유지 시간은 YAML 파일에서 Parameter 로 전달받는다. 설정된 시간이 지나면 다음 신호로 변경하고, 현재 신호 상태를 Topic 으로 Publish 한다.

2.2 Vehicle Node

vehicle_node 는 차량의 생성과 이동을 담당한다.

traffic_light_node 가 발행하는 신호 상태 Topic 을 Subscribe 하고, 수신한 신호에 따라 차량의 동작을 결정한다.

- GREEN: 설정된 속도로 이동
- YELLOW: 감속하거나 정지 준비
- RED: 정지

차량의 이동 속도는 YAML 파일에서 Parameter 로 전달받는다. 계산된 차량 위치는 별도의 Topic 으로 Publish 하여 Qt GUI 가 사용할 수 있도록 하였다.

2.3 Qt GUI

Qt GUI 는 신호등과 차량의 현재 상태를 시각적으로 표현한다.

Qt 에서는 신호 변경 시간이나 차량 이동 위치를 직접 계산하지 않는다. ROS2 Topic 을 통해 전달받은 현재 신호 상태와 차량 위치를 화면에 표시하는 기능만 담당하도록 구성하였다.

이를 통해 시뮬레이션 로직과 화면 표시 기능을 서로 분리하였다.

3. Node 및 Topic 통신 구조

프로그램에서 사용한 Node 와 Topic 은 다음과 같다.

송신 Node	Topic	메시지 타입	수신 Node	전달 데이터
traffic_light_node	[신호 Topic 이름]	[메시지 타입]	vehicle_node	현재 신호 상태
traffic_light_node	[신호 Topic 이름]	[메시지 타입]	Qt GUI	현재 신호 상태
vehicle_node	[위치 Topic 이름]	[메시지 타입]	Qt GUI	현재 차량 위치

traffic_light_node 가 현재 신호를 Publish 하면 vehicle_node 와 Qt GUI 가 동일한 Topic 을 Subscribe 한다.

vehicle_node 는 신호 상태를 이용하여 차량의 이동 여부를 결정하고, 계산된 차량 위치를 Publish 한다. Qt GUI 는 차량 위치를 Subscribe 하여 화면에 차량을 표시한다.

4. 신호등 상태 관리

신호등은 상태를 나타내는 변수를 이용하여 구현하였다.

프로그램이 시작되면 초기 신호는 [RED 또는 실제 초기 상태]로 설정하였다. Timer Callback 이 실행될 때 현재 신호의 경과 시간을 확인하고, YAML 에서 설정한 시간이 지나면 다음 신호로 변경하였다.

상태 변경 과정은 다음과 같다.

1. RED 상태가 [red_duration]초 동안 유지된다.
2. 시간이 지나면 GREEN 상태로 변경된다.
3. GREEN 상태가 [green_duration]초 동안 유지된다.
4. 시간이 지나면 YELLOW 상태로 변경된다.
5. YELLOW 상태가 [yellow_duration]초 동안 유지된다.
6. 시간이 지나면 다시 RED 상태로 변경된다.

신호가 변경될 때마다 현재 상태를 [신호 Topic 이름] Topic 으로 Publish 하였다.

5. 차량 이동 처리

차량은 현재 신호 상태와 YAML 에서 전달받은 속도 값을 이용하여 이동한다.

GREEN 신호를 수신하면 Timer 가 실행될 때마다 차량 위치에 속도에 따른 값을 더하였다. RED 신호를 수신하면 차량 위치를 변경하지 않아 정지하도록 하였다.

YELLOW 신호에서는 [본인이 구현한 방식: 속도를 줄임/정지/정지선까지 이동]하도록 구현하였다.

차량 위치의 계산은 Qt 가 아니라 vehicle_node 에서 처리하였다. 계산된 위치는 [위치 Topic 이름] Topic 으로 Publish 하였다.

차량이 화면의 끝에 도착한 경우에는 [처음 위치로 이동/차량 제거 후 재생성/실제 구현 방식]하도록 처리하였다.

6. YAML Parameter 구성

신호등의 점등 시간과 차량 속도를 코드 밖에서 변경할 수 있도록 YAML 파일을 작성하였다.

```
traffic_light_node:
  ros__parameters:
    red_duration: [빨간불 시간]
    green_duration: [초록불 시간]
    yellow_duration: [노란불 시간]

vehicle_node:
  ros__parameters:
    vehicle_speed: [차량 속_속도 값]
```

traffic_light_node에서는 빨간불, 초록불 및 노란불의 지속 시간을 Parameter로 선언하였다.

vehicle_node에서는 차량의 이동 속도를 Parameter로 선언하였다.

YAML 파일에 작성한 Node 이름은 Launch 파일에서 설정한 Node 이름과 일치하도록 작성하였다.

Parameter를 적용한 결과, 소스 코드를 수정하지 않고도 신호등의 점등 시간과 차량 속도를 조절할 수 있었다.

7. Launch 파일 구성

전체 시스템을 하나의 명령으로 실행하기 위해 Launch 파일을 작성하였다.

Launch 파일에는 다음 실행 항목을 포함하였다.

1. traffic_light_node 실행
2. vehicle_node 실행
3. Qt GUI Node 실행
4. YAML Parameter 파일 전달

각 Node의 parameters 항목에는 동일한 YAML 설정 파일을 전달하였다. YAML 파일에서 Node 이름을 기준으로 필요한 설정이 각각 적용되도록 구성하였다.

실행 명령은 다음과 같다.

```
ros2 launch [패키지 이름] [Launch 파일 이름].launch.py
```

Launch 파일을 이용하여 여러 터미널에서 Node를 각각 실행할 필요 없이 전체 시뮬레이터를 한 번에 실행할 수 있었다.

8. Qt 화면 구성

Qt 화면에는 다음 요소를 표시하였다.

- 현재 신호등 상태
- 빨간불, 노란불, 초록불 표시
- 도로 또는 교차로
- 차량
- [추가로 표시한 정보: 차량 위치, 현재 속도 등]

현재 신호가 RED 이면 빨간색 신호를 활성화하고, GREEN 이면 초록색 신호를 활성화하였다. YELLOW 상태에서는 노란색 신호가 표시되도록 구현하였다.

차량의 위치는 `vehicle_node` 가 발행한 값을 이용하여 화면 좌표를 변경하였다.

Qt에서는 차량의 다음 위치나 신호 전환을 계산하지 않고, ROS2 Node 가 Publish 한 결과만 화면에 반영하였다.

9. 결과 및 고찰

이번 과제를 통해 여러 Node 가 Topic 을 이용하여 하나의 시스템처럼 동작하는 과정을 이해할 수 있었다.

신호등 상태 관리, 차량 이동 계산, Qt 화면 표시 기능을 각각 분리하여 구현하였다. 각 기능을 별도의 Node 로 분리한 결과 한 부분을 수정하더라도 다른 기능에 미치는 영향을 줄일 수 있었다.

또한 Qt 에서 모든 기능을 처리하지 않고 ROS2 Node 가 계산한 결과만 표시하도록 구성하면서 프로그램의 로직과 GUI 의 역할을 분리할 수 있었다.

YAML Parameter 를 이용하여 신호 유지 시간과 차량 속도를 코드 외부에서 변경할 수 있었다. 이를 통해 설정값을 변경할 때마다 소스 코드를 수정하는 방식보다 유지보수와 테스트가 편리하다는 것을 알게 되었다.

Launch 파일을 이용하여 여러 Node 와 YAML 설정을 하나의 명령으로 실행하는 방법도 익힐 수 있었다.